Software Design Document (SDD)

Team-08

Plurality / STV Voting Algorithms

Software Design Document

Name(s):

Nicholas Nguyen
James Lu (lu000605)
Finnian Gidden (gidde032)
Carter Randall (randa464)

Date: (3/6/26)

TABLE OF CONTENTS

1. INTRODUCTION 2

1.1 Purpose 2

1.2 Scope 2

1.3 Overview 2

1.4 Reference Material 2

1.5 Definitions and Acronyms 2

2. SYSTEM OVERVIEW 2

3. SYSTEM ARCHITECTURE 2

3.1 Architectural Design 2

3.2 Decomposition Description 3

3.3 Design Rationale 3

4. DATA DESIGN 3

4.1 Data Description 3

4.2 Data Dictionary 3

5. COMPONENT DESIGN 3

6. HUMAN INTERFACE DESIGN 4

6.1 Overview of User Interface 4

6.2 Screen Images 4

6.3 Screen Objects and Actions 4

7. REQUIREMENTS MATRIX 4

8. APPENDICES 4

1. INTRODUCTION

1.1 Purpose

This software document describes the architecture and system design of the plurality and STV voting systems for the developers working on the project.

1.2 Scope

The software is meant to implement the STV and plurality voting algorithms. We are not to validate the input, and the input is assumed to be reliable. The project will output the winner of the election and settle any ties.

1.3 Overview

This software design document describes the plurality and STV voting systems, its system architecture, how the data is designed, and its presentation to users. Multiple diagrams are made to describe the system and are explained in detail.

2. SYSTEM OVERVIEW

The system is a voting system that is designed to perform plurality and STV voting systems on user input candidate info and ballot csv files. The information that is required by the user is the command choice, number of seats, algorithm (STV/plurality), candidate csv file, and ballot file(s). This is handled by prompting the user to input this information through the user interface. For the plurality voting system the ballot is required to have only one candidate indicated as the choice for the vote. Each ballot will count as one vote for that candidate. The candidate with the greatest number of votes will be declared the winner unless there's a tie then a coin toss will be used to determine the winner. This is handled in the system by parsing the candidate and ballot data into objects, then assigning each ballot to a candidate and assigning a count to each candidate. After each ballot is counted the final counts will be checked to see if there is a tie, if there is no tie the winner will be the candidate with the highest count. If there is a random coin toss will be used to break the tie and determine the winner between the tied candidates. For the STV algorithm each ballot must have at least 60% of the candidates ranked. All of the ballots will be shuffled and the droop quota is determined. Then all the ballots are processed until all candidates are sorted between elected and non-elected lists in order of elected/eliminated on respective lists. Once the election is complete a report can be generated with important information about the

election. To ensure all of this occurs in the system we have the droop quota determined and the shuffling of ballots accounted for in the STV subclass with all of the other relevant information for the report stored in the election interface.

3. SYSTEM ARCHITECTURE

3.1 Architectural Design

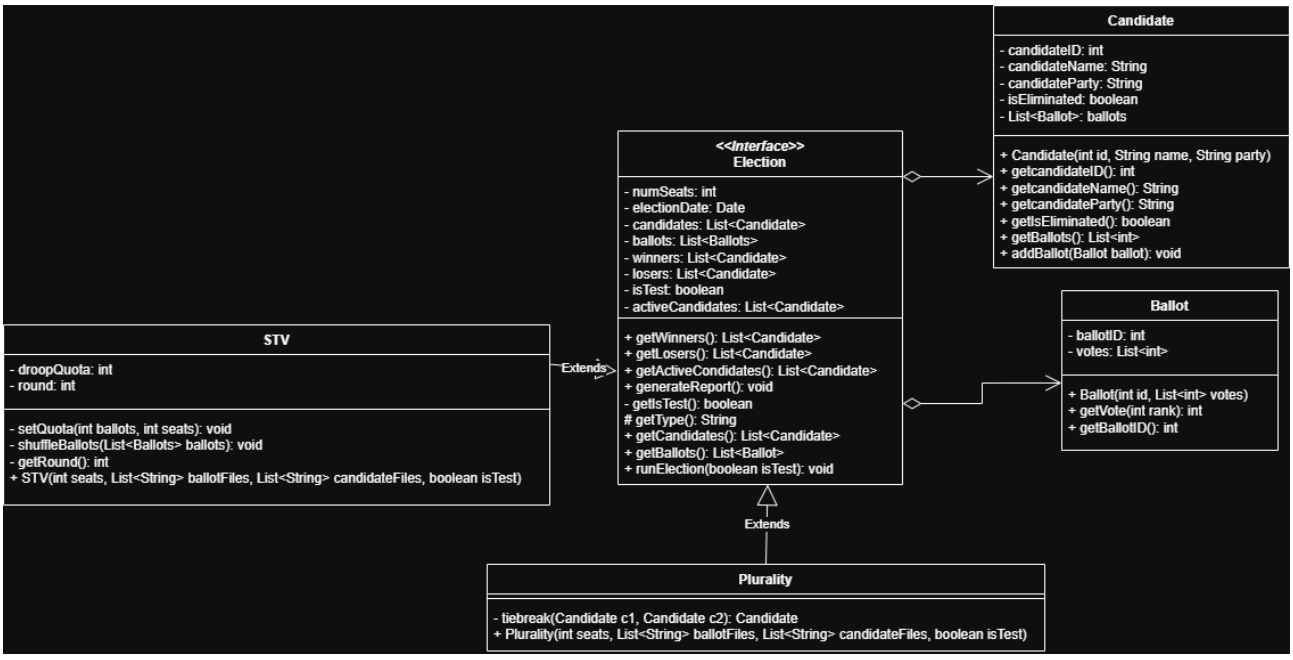


Figure 3.1.1, UML Class Diagram

There are two subsystems, one for STV voting and the other for Popularity voting. Both utilize the same Candidate class and the same Ballot class. As both of the subsystems also contain a test option, each of them handles two types of elections (STV with/without test, Plurality with/without test), covering any type of election the user may want to calculate. The subsystems also cover the case where the user needs help navigating the system through the help function. The Ballot and Candidate classes turn the user-provided inputs into an organized format and make them compatible with the subsystems. The getter functions in the classes make it easy and intuitive for the subsystems to access information and execute the election. Each election can easily “contain” each of its relevant ballots and candidates in lists.

Election data, functions: The Election abstract class contains the bulk of the functions for the STV and Plurality subsystems. *Simple getters and setters are not described*.

- int numSeats: number of seats inputted by the user
- Date electionDate: date of the election
- List<Candidate> candidates: A list containing all of the candidates that participated in the election as Candidate objects
- List<Ballot> ballots: A list containing all of the ballots as Ballot objects
- List<Candidate> winners: A dynamically updated list containing the elected group of candidates
- List<Candidate> losers: A dynamically updated list containing the non-elected group of candidates
- boolean isTest: A boolean that is true if the user requests a test election, and false otherwise
- List<Candidate> activeCandidates: A dynamically updated list containing the candidates that have not been eliminated; at the beginning of an election, it is the same as the list “candidates”.
- void generateReport(): Creates a formatted report document of the entire election. The function prints a quick summary of the election to the user interface and tells the user where the report document is stored
- void runElection(boolean isTest): Execute the election by counting the ballots and finding a winner(s). This is executed after the user has been prompted for election information. This function calls generateReport() at the end of its runtime.

STV data, functions: The STV class extends Election and adds a few extra functions and attributes.

- int droopQuota: The quota to move a candidate to the elected list, calculated using the Droop function
- int round: The current round the election is processing
- void setQuota(int ballots, int seats): Calculate the Droop quota using the equation floor(ballots/[seats + 1]) + 1 and set droopQuota to that value
- void shuffleBallots(List<Ballots> ballots): Shuffle the list “ballots” using a built in Java library function
- STV(int seats, List<String> ballotFiles, List<String> candidateFiles, boolean isTest): A constructor for an STV object that takes in inputs corresponding to the user input. Initializes all data into Ballot and Candidate objects. Initialize winners and losers as empty lists.

Plurality data functions: The Plurality class also extends Election and adds a few extra functions.

- tiebreak(Candidate c1, Candidate c2): Performing a fair coin-flip tiebreaker between c1 and c2, returning the winner
- Plurality(int seats, List<String> ballotFiles, List<String> candidateFiles, boolean isTest): A constructor for a Plurality object that takes in inputs corresponding to the user input. Initializes all data into Ballot and Candidate objects. Initialize winners and losers as empty lists.

Ballot data functions: The Ballot class is an object representation of one line in the provided ballot CSV data files.

- int ballotID: The numerical ID of the ballot
- List<int> votes: The candidate(s) the voter voted for. The List has the length of the number of candidates. If a candidate was not voted for, the value for their entry would be 0. In STV, the ordering for each candidate would be 1, 2, 3…etc. Each candidate is assigned an index based on the first line of the CSV. For example, if the CSV had the first line “ballotID, 123, 456”, the candidate with id “123” would correspond to index 0 in the list, and “456” would correspond to index 1.
- Ballot(int id, List<int> votes): Creates a ballot, setting ballotID and votes from the passed parameters accordingly.

Candidate data functions: The Candidate class is an object representation of one line in the provided candidate CSV data files. Additionally, each Candidate object tracks how many ballots are assigned to it.

- int candidateID: The numerical ID of the candidate (first value in a CSV)
- String candidateName: The name of the candidate (second value in a CSV)
- String candidateParty: The party of the candidate (third value in a CSV)
- isEliminated: True if the candidate has been added to the non-elected list, false otherwise
- List<Ballot> ballots: A list storing the Ballot objects that vote for the candidate
- Candidate(int id, String name, String party): A constructor for a Candidate object, initializing candidateID, candidateName, and candidateParty to custom values. isEliminated is initialized to false, ballots is initialized as an empty list
- void addBallot(Ballot ballot): add a Ballot object to the end of the list ballots.

3.2 Decomposition Description

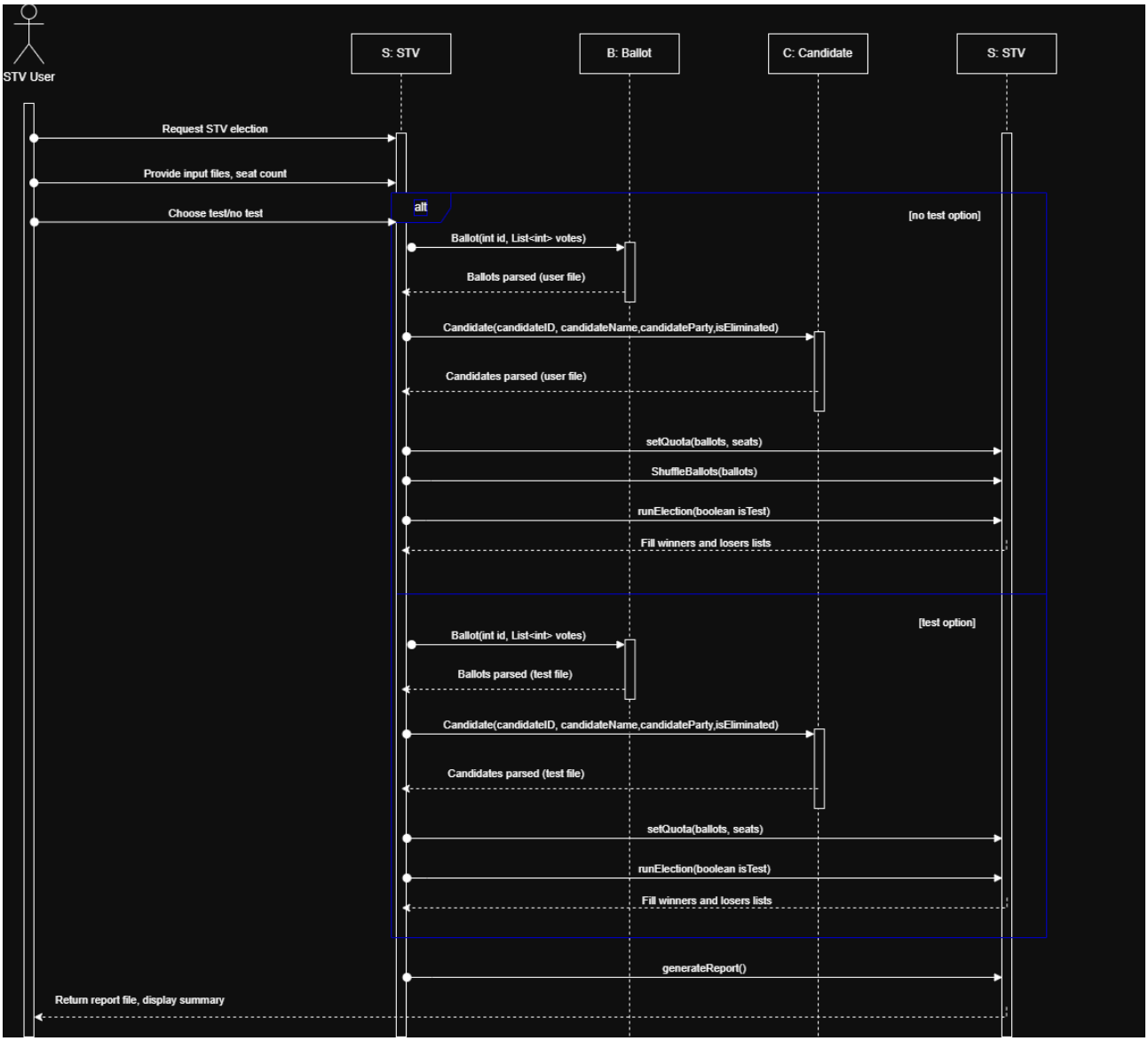


Figure 3.2.1, STV Sequence Diagram

The STV sequence diagram begins with the user requesting an STV election and providing the required files. The user then chooses whether or not they want a test option. If no test is requested, the STV subsystem parses the candidate and ballot information from the user-provided files, calculates the quota, shuffles the ballots, and runs the election. However, if a test is requested, the subsystem instead parses from a pre-existing test file and does not shuffle the ballots, but still calculates the quota and runs the election. In both cases, a report file is generated and returned, and a summary is displayed to the user.

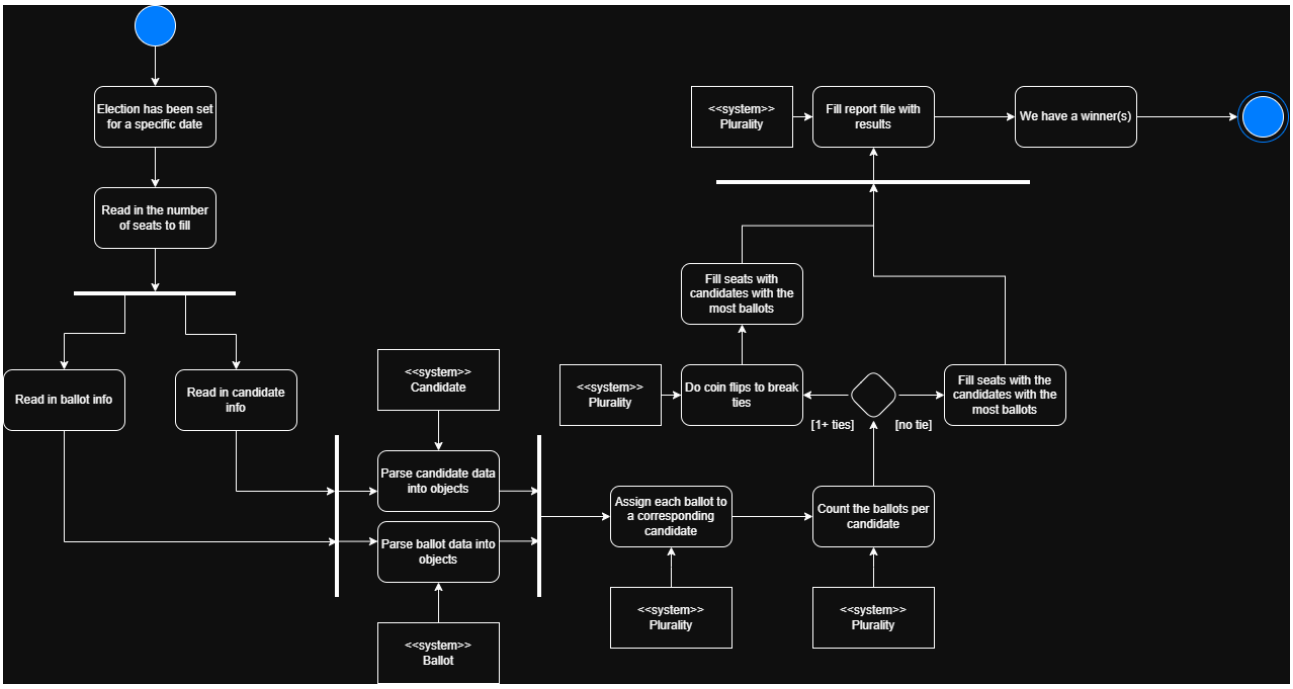


Figure 3.2.2, Popularity/Plurality Activity Diagram

The Popularity/Plurality diagram begins by taking in the required information from the user. Following that, the system turns said information into Candidate and Ballot objects, which are compatible with the Plurality system. The system then assigns the candidate objects their corresponding ballot objects and iterates through all of the candidate objects to check which ones have the most ballots. The system will then pick a winner/winners and perform tiebreaks if necessary. The election results and specifications are then compiled into a file for the user, and a winner/winners is declared.

3.3 Design Rationale

While STV voting and Plurality voting use ballots differently, both voting systems use the same ballot class, as an STV ballot can be repurposed into a popularity ballot where only one candidate is ranked. This makes the design simpler and easier to understand. STV and Plurality extend from an Election abstract class as they share many of the same variables and only differ in a few (i.e., droopQuota, coinToss). Most of their functions also have the same name but require a different implementation. While it may seem redundant that candidates have an isEliminated attribute while the winners and losers lists are present in the Election classes, this attribute makes it quicker to determine if a candidate is eliminated through a getter vs. traversing a list. The winners and losers lists are mainly for printing out the results.

4. DATA DESIGN

4.1 Data Description

The system handles two types of information: candidate data and ballot data. This data is sourced from input files provided by the user and stored in several ways, outlined below. All data is loaded into memory at the start of an election and results are written to the console and sent to a report output file after the election completes; no permanent databases are used by the system at any point.

Upon starting the program, the user provides the names of a candidate file and one or more ballot files. The system reads these files and instantiates Candidate and Ballot objects for each respective candidate and ballot. These objects are then passed into either an STV or plurality election subsystem, which uses the two sets of objects to determine election winners. Finally, the results are written to the console, and an output file with election results is generated for STV elections.

Data Storage Items

Below is a list of the various entities and types of data stored in the system, how they are stored, and their descriptions.

Data Entity	Data Type	Storage Mechanism	Description
Candidate	Object (Class)	Runtime Memory	Represents a candidate. Candidates are instantiated from each line in the candidate input file.
Ballot	Object (Class)	Runtime Memory	Represents a voter's ballot. Ballots are instantiated from each line in the ballot input files.
Candidate File	Plain Text / CSV	File System (Input)	Text file listing candidate names and pertinent information, one per line.
Ballot File(s)	Plain Text / CSV	File System (Input)	One or more files, each containing ballots with voter rankings (STV) or vote assignments (plurality).
Election Results	Plain Text	Console (Output)	Generated at the end of every election. Displays relevant election information and reveals the results.

Election Report	Plain Text	File System (Output)	Outfile generated at the end of each election. Contains relevant election information, and in-depth data on ballot distribution and candidates' statuses through the election for STV elections.
Election State	Primitive Variables	Runtime Memory	Includes seat count, algorithm type, round number (STV), and running vote totals.

Candidate Data

Candidate data is sourced from the user-provided candidate file, where each line of the file corresponds to one candidate. Upon loading, the system creates a Candidate object for each entry in the file. Each Candidate object stores the candidate's unique identifier, name, party, associated ballots, and whether they have been eliminated. This elimination flag matters, as in the STV subsystem, candidates are eliminated in batches across rounds, and their ballot objects are redistributed to the next-ranked candidate that is not eliminated. In the plurality subsystem, each candidate simply accumulates a total vote count and the totals are compared once all ballots are processed.

Ballot Data

Ballot data is sourced from one or more ballot files provided by the user. Each ballot file may contain multiple individual ballots. In the plurality algorithm, each ballot records a single candidate choice, whereas in the STV algorithm, each ballot records a ranked choice of the voters' top five candidates. After processing the ballot file, the system creates a Ballot object for each individual ballot entry in the file. Finally, Ballot objects are assigned to the corresponding Candidate object for processing, the execution of which varies between the STV and plurality algorithms.

Election Output Data

After each election is completed, the system displays to the console plain text results of the election. This output includes varying information depending on the election type (STV or plurality), but always includes the election type, number of seats contested and candidates, the total number of ballots cast, the winner or winners, and pertinent information about the elected and non-elected candidates depending on the type of election. An outfile is also generated with an election report, containing further election information and, for STV

elections, detailed round-by-round information on Ballot object assignment and Candidate object status.

4.2 Data Dictionary

Access modifier key: + public - private # protected

Class: Ballot

The Ballot class is a simple data-holding class used by both the STV and plurality algorithms, representing a single voter's ballot. Ballot objects are instantiated from the ballot input files provided by the user at the start of each election, and read in line-by-line by the system.

Attributes

Modifier	Attribute	Type	Description
-	ballotID	int	A unique integer ID provided by the ballot input file.
-	votes	List<int>	An ordered list of candidate IDs representing the voter's ranked preferences. In plurality elections this list contains just one entry, but, in STV elections the list is ordered from most preferred to least preferred.

Methods

Modifier	Method	Parameters	Return Type	Description
+	Ballot(int id, List<int> votes)	**id (int):** Unique ballot identifier **(List<int>):** Ordered list of candidate IDs	-	Constructor. Instantiates a Ballot with the given ID and vote list.
+	getBallotID(): int	None	int	Getter. Returns the unique ballot identifier.
+	getVote(int rank): int	rank (int): The zero-based desired rank	int	Getter. Returns the candidate ID at the given rank position in the votes list.

Class: Candidate

The Candidate class represents a single candidate in any election. Candidate objects are instantiated from the candidate input file provided by the user at the start of each election, and are used in both the STV and plurality algorithms.

Attributes

Modifier	Attribute	Type	Description
-	candidateID	int	A unique integer ID provided by the candidate input file.
-	candidateName	String	The candidate's full name, provided by the candidate input file.
-	candidateParty	String	The candidate's party affiliation, provided by the candidate input file.
-	isEliminated	boolean	A flag indicating whether a candidate has been eliminated from the election. Used in STV elections, and is set to true when a candidate is eliminated.
-	ballots	List<int>	A list of ballot IDs representing the ballots currently assigned to this candidate. Used in STV elections, and is redistributed upon candidate elimination.

Methods

Modifier	Method	Parameters	Return Type	Description
+	Candidate(int id, String name, String party)	**id (int):** Unique candidate identifier **(String):** Candidate's full name **(String):** Candidate's party affiliation	-	Constructor. Instantiates a Candidate with the given ID, name, and party. Initializes ballots to an empty list and isEliminated to false.
+	getCandidateID(): int	None	int	Getter. Returns the unique candidate identifier.
+	getCandidateName(): String	None	String	Getter. Returns the candidate's full name.
+	getCandidateParty(): String	None	String	Getter. Returns the candidate's party affiliation.
+	getIsEliminated(): boolean	None	boolean	Getter. Returns a boolean representing whether the candidate is eliminated.
+	getBallots(): List<int>	None	List<int>	Getter. Returns the list of ballot IDs currently assigned to this candidate.
+	addBallot(): void	None	void	Adds a ballot ID to this candidate's ballot ID list.

Interface: Election

The election interface defines the outline for all election types in the system. Both the STV and plurality classes implement this interface and inherit its attributes and method signatures.

Attributes

Modifier	Attribute	Type	Description
-	numSeats	int	The number of seats to be filled in this election, provided by the user during the “run election” command sequence.
-	electionDate	Date	The date on which the election is run.
-	candidates	List<Candidate>	The complete list of all Candidate objects participating in the election, provided by the candidate input file.
-	ballots	List<Ballot>	The complete list of all Ballot objects cast in the election, provided by the ballot input file(s).
-	winners	List<Candidate>	An ordered list of elected Candidate objects; the order reflects the order in which candidates were elected.
-	losers	List<Candidate>	An ordered list of not elected Candidate objects; the order reflects the order in which candidates were eliminated.
-	isTest	boolean	Flag indicating whether the election is running in test mode.
-	activeCandidates	List<Candidate>	A round-by-round updating list of candidates who have not yet been eliminated or elected in the current round.

Methods

Modifier	Method	Parameters	Return Type	Description
+	runElection(boolean isTest): void	**isTest (boolean):** If true, runs in test mode	void	Executes the full election algorithm including ballot assignment, vote counting, and result determination. After completion, displays the post-election results summary containing election type, number of ballots, number of seats, number of candidates, elected candidates with relevant statistics, and non-elected candidates with relevant statistics.
+	generateReport(): void	None	void	Generates the election report containing pertinent statistics and writes it to an output text file. For STV elections, the report shows candidate’s ballot assignments after each round and the shuffled ballot order, if shuffle was enabled.
+	getCandidates(): List<Candidate>	None	List<Candidate>	Getter. Returns the list of all candidates in the election.

+	getActiveCandidates(): List<Candidate>	None	List<Candidate>	Getter. Returns the list of active candidates in the election.
+	getWinners(): List<Candidate>	None	List<Candidate>	Getter. Returns the ordered list of elected candidates.
+	getLosers(): List<Candidate>	None	List<Candidate>	Getter. Returns the ordered list of non-elected candidates.
+	getBallots(): List<Ballot>	None	List<Ballot>	Getter. Returns the list of ballots cast in the election.
-	getIsTest(): boolean	None	boolean	Getter. Returns whether the election is being run in test mode.
#	getType(): String	None	String	Getter. Returns the election type as a string.

Class: STV (extends Election)

The STV class implements the Election interface and represents a Single Transferable Vote election. STV objects are instantiated by the system using runtime variables and text files provided by the user at the start of each election.

Attributes (in addition to those inherited from Election)

Modifier	Attribute	Type	Description
-	droopQuota	int	The Droop quota threshold, calculated using the floor of totalBallots / (numSeats + 1)) + 1. Signifies the number of votes needed by a candidate to become elected.
-	round	int	Tracks the current election round number; initialized to 1.

Methods (in addition to those inherited from Election)

Modifier	Method	Parameters	Return Type	Description
+	STV(int seats, List<String> ballotFiles, List<String> candidateFiles, boolean isTest)	**seats (int):** Number of seats **ballotFiles (List<String>):** Ballot input filenames **candidateFiles (List<String>):** Candidate input filename	-	Constructor. Instantiates the STV election object by loading candidates and ballots from the user-provided files, setting the number of seats, and toggling shuffle to false if run in test mode.

		isTest (boolean): Test mode flag		
-	setQuota(int ballots, int seats): void	**ballots (int):** Total number of ballots cast **seats (int):** Number of seats to fill	void	Computes and stores the election's Droop quota.
-	shuffleBallots(List<Ballot > ballots): void	**ballots (List<Ballot>):** The ballot list to shuffle	void	Randomly reorders the ballot list before the first round of the election.
-	getRound(): int	None	int	Getter. Returns the current round number.

Class: Plurality (extends Election)

The Plurality class implements the Election interface and represents a plurality election. Plurality objects are instantiated by the system using runtime variables and text files provided by the user at the start of each election.

Methods (in addition to those inherited from Election)

Modifier	Method	Parameters	Return Type	Description
+	Plurality(int seats, List<String> ballotFiles, List<String> candidateFiles, boolean isTest)	**seats (int):** Number of seats ballotFiles **(List<String>):** Ballot input filenames **candidateFiles (List<String>):** Candidate input filename **isTest (boolean):** Test mode flag	-	Constructor. Instantiates the plurality election object by loading candidates and ballots from the user-provided files, and setting the number of seats.
-	tiebreak(Candidate c1, Candidate c2): Candidate	**c1 (Candidate):** First tied candidate **c2 (Candidate):** Second tied candidate.	Candidate	Resolves a tiebreaker between candidates using the coinToss attribute. Returns the winning candidate.

5. COMPONENT DESIGN

The following is pseudocode for the functions/classes described in 3.2. *Note that pseudocode for getters is not provided due to the simplicity of the functions.*

Ballot(int id, List<int> votes)

Set ballotID to id

Store voting information (votes) to votes

Return the Ballot

Candidate(int id, String name, String party)

Set candidateID to id, candidateName to name, candidateParty to party

Initialize isEliminated to False

Initialize a list to store ballots for the candidate

Return the Candidate

STV.setQuota(ballots, seats)

Calculate quota with Droop equation

Set STV.droopQuota to the quota

STV.shuffleBallots()

Shuffle STV.Ballots with a Java library function

STV.runElection(boolean isTest)

Set STV.numSeats to specified seat number, Set STV.electionDate to relevant date

If isTest

Parse Ballots from test file

Add Ballots to STV.Ballots

Parse Candidates from test file

Add Candidates to STV.candidates, STV.activeCandidates

```
Else
        If file cannot be found
                Return error “invalid CSV file”
        Parse Ballots from user-provided file
        Add Ballots to STV.Ballots
        Parse Candidates from user-provided file
        Add Candidates to STV.candidates, STV.activeCandidates
Set STV.round to 0
While True
        If size of STV.winners == STV.numSeats // Election is done
                STV.generateReport()
                Return
        // Leftover candidates, pull winners from non-elected
        Else if size of STV.winners != STV.numSeats AND STV.activeCandidates is
empty
                While size of STV.winners < STV.numSeats, STV.losers not empty
                        Add a Candidate C to STV.winners from back of STV.losers
                        Remove C from STV.losers
                STV.generateReport()
                Return
        Clear C.ballots for all candidates C
        // Iterate and assign ballots
        For all Ballots B in STV.Ballot
                For each vote V in B.votes in ascending order
                        If B.V’s corresponding candidate is in STV.losers
```

```
                                or NOT in STV.activeCandidates
                                    Check next-highest-ranked candidate
                                Else // Valid candidate (Valid) for this ballot found
                                    Break from this loop
                            Add B to Valid.ballots
                            For Each Candidate C in STV.activeCandidates
                                If size of C.getBallots() >= STV.droopQuota
                                    Add C to STV.winners, Remove from
                                STV.activeCandidates
                                    Remove all ballots from C.ballots
                                    Break from outer For loop (But NOT while loop)
                        If size of STV.winners did not increase this iteration
                            Create Least Vote Candidate L with INF ballots
                            For Each Candidate C in STV.activeCandidates
                                If size of C.Ballots < size of L.ballots
                                    L = C
                            Add L to STV.losers, Remove from STV.activeCandidates
                            Remove all ballots from C.ballot
                        STV.round++


Election.generateReport()
        Print simplified summary of election results to user interface
        Write("SRS Election")
        Write(STV.electionDate)
        For all Candidates C in STV.candidates
```

```
            Write(C.candidateName, C.candidateParty)
        Write(Size of STV.candidates)
        Write(STV.numSeats)
        Write("Elected")
        For each C from front to back in STV.winners
            Write(C.name)
        Write("Non-elected")
        For each C from back to front in STV.losers
            Write(C.name)
        Save file to directory
        Print("Summary file can be found in *directory*")

Plurality.runElection(boolean isTest)
        Set STV.numSeats to specified number of seats, Set STV.electionDate to relevant
date
        If isTest
            Parse Ballots from test file
            Add Ballots to Plurality.Ballots
            Parse Candidates from test file
            Add Candidates to Plurality.candidates
        Else
            If file cannot be found
                Return error "invalid CSV file"
            Parse Ballots from user-provided file
            Add Ballots to Plurality.ballots
```

```
        Parse Candidates from user-provided file
        Add Candidates to Plurality.candidates
    For all Ballots B in Plurality.ballots
        Set candidateID = B.getVote(0) // Only need first choice vote
        Find Candidate C in Plurality.candidates with candidateID
        Add B to C.votes
    Sort Plurality.Candidates in ascending order
    Int i = 0 // Tracker
    While size of Plurality.winners < Plurality.numSeats
        Add Plurality.candidates[i] to Plurality.winners
        i++
    Plurality.generateReport()
    Return
```

6. HUMAN INTERFACE DESIGN

6.1 Overview of User Interface

The voting software system uses a text-based command-line interface and all interaction between the user and the system occurs through a terminal or console window. The interface is driven by user prompts and sequential; first, the system displays a prompt requesting a command input, then the user types their response, and finally the system processes the input and either requests further information relevant to the election, displays feedback in the case of help requests or invalid commands, or displays election results and produces a report outfile once an election is successfully ran.

The user interface includes three initial commands:

- run election: Initiates the process of configuring elections and executing the election workflow.
- help: Displays a help screen with command descriptions and instructions for program use.

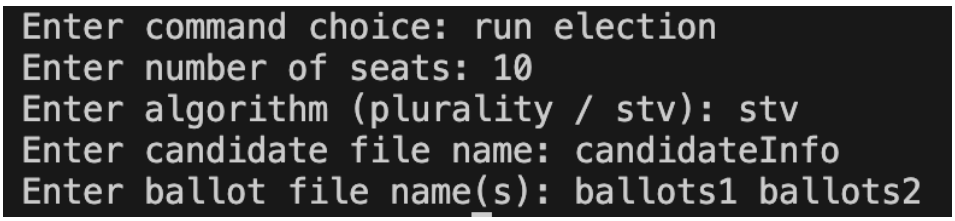
- exit: Terminates the program with a goodbye message.

After launching the program, the user is immediately presented with the main command prompt in the console window. If the user enters the run election command, the system collects the additional required parameters in a predetermined sequence. If the user enters an invalid command, the system outputs an error message describing the issue, and displays the main command prompt once more so the user can try again. If the user continues to enter invalid prompts, this loop continues until the user enters a valid command or exits.

6.2 Screen Images

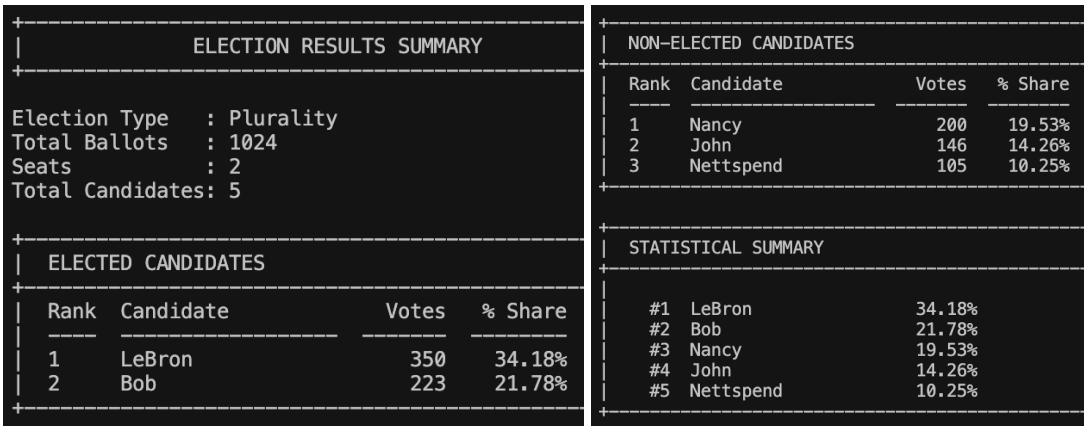
Inputting Arguments (Run Election):

The following console image shows the sequence of prompts displayed when the user inputs the command “run election”. The user is asked, in order, for: the number of seats, the algorithm type (plurality or STV), the candidate file name, and one or more ballot file names.



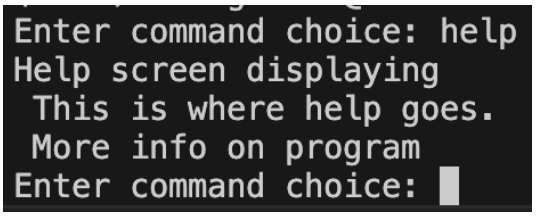
Displaying Election Results:

The following console image illustrates how the system outputs the results after a user has successfully run an election. The program displays a summary of the election and pertinent information about candidates to the user, and then returns to the main command prompt allowing the user to run another election if desired.



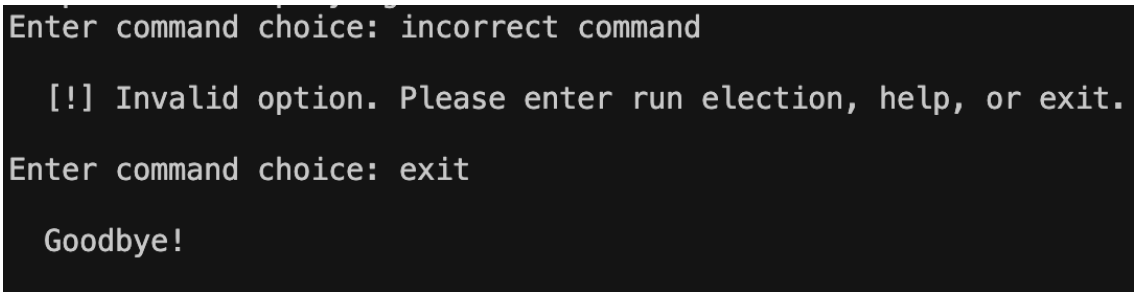
Requesting Help:

The following console image shows the system's response when the user inputs the command “help”. The program displays the help screen content (placeholder used below) and then shows the user the main command prompt once more.



Entering an Invalid Command and Exiting:

The following console image shows the system's response when the user inputs an invalid command and exits. The system outputs an error message identifying the valid options and then displays the main command prompt again for the user. Below, the exit command is used immediately after, whereupon the program terminates and outputs a goodbye message.



6.3 Screen Objects and Actions

Screen Object	Type	Action / Behavior
Command Prompt (Enter command choice:)	Input Prompt	Displayed at program startup and after each action outside of exit (help, invalid command, or election completion). Waits for the user to input a command and press ‘Enter’. Accepts the commands “run election”, “help”, and “exit”. Any other input outputs an error message and the system returns to this prompt.
Seat Count Prompt (Enter number of seats:)	Input Prompt	Displayed immediately after “run election” is entered. Waits for the user to type a positive integer and press ‘Enter’. Non-integer/non-positive inputs output an error and re-prompts, while valid input is stored as the numSeats parameter.
Algorithm Prompt (Enter algorithm (plurality / stv):)	Input Prompt	Displayed after a valid seat count is entered. Waits for the user to type either “plurality” or “stv” and press ‘Enter’. Input is case-insensitive and invalid input outputs an error and re-prompts, while valid input determines which election subsystem is used.
Candidate File Prompt (Enter candidate file name:)	Input Prompt	Displayed after a valid algorithm is selected. Waits for the user to type the name of the candidate data file and press ‘Enter’. The system confirms that the file exists and is readable, if the file is invalid or missing, the system outputs an error message and re-prompts.
Ballot File Prompt (Enter ballot file name(s):)	Input Prompt	Displayed after a valid candidate file is provided. Waits for the user to type one or more ballot file names, separated by spaces, and press ‘Enter’. Each file is validated independently, invalid file names produce specific error messages, while valid files are loaded into the system and processed.
Help Screen	Output Text	Displayed in response to the help command. Shows the program name, available commands, and their descriptions. Non-interactive, and after display the system returns to the main command prompt.
Error Message ([!] Invalid option...)	Output Text	Displayed whenever the user enters an invalid command to the main prompt. Informs the user of the valid options and returns to the main command prompt.

Results Confirmation Message	Output Text	Displayed after an election is successfully run. Confirms that the election was a success and displays the results, also generating an output file containing an election report. The system then returns to the main command prompt, allowing the user to run another election or exit.
Goodbye Message (Goodbye!)	Output Text	Displayed in response to the exit command. Confirms that the program has terminated, and the program is closed making further input impossible.

All screen objects are displayed as plain text in the console. There are no graphical elements, menus, or any ways for the user to interact with the system outside of terminal prompts, making the interface entirely keyboard-driven.

7. REQUIREMENTS MATRIX

Requirement ID	Requirement Description	Data associated and its use
UC_001	User wants plurality voting without testing	Classes: Election, Plurality Methods: tiebreak, Plurality, runElection Uses of each: The user uses the Election Interface to indicate that they want to use the plurality algorithm without testing (!isTest attribute). Then, tiebreaking and Plurality are used when running the election algorithm.
UC_002	User wants STV voting without testing	Classes: STV, Election Methods: STV, getRound, shuffleBallots, setQuota, runElection

		Uses of each: The user indicates they want to run an STV election without testing (!isTest attribute) to the Election interface, then STV, shuffleBallots, setQuota, and getRound, are used in the process of running the STV election algorithm.
UC_003	User wants plurality voting with testing	Classes: Election, Plurality Methods: tiebreak, Plurality, runElection Uses of each: The user uses the Election Interface to indicate that they want to use the plurality algorithm with testing (isTest attribute). Then, tiebreaking and Plurality are used when running the election algorithm.
UC_004	Users wants STV voting with testing	Classes: STV, Election Methods: STV, getRound, shuffleBallots, setQuota, runElection Uses of each: The user indicates they want to run an STV election with testing (isTest attribute) to the Election interface, then STV, shuffleBallots, setQuota, and getRound, are used in the

		process of running the STV election algorithm.
UC_005	User accesses audit report of STV election	Classes: Election Methods: generateReport Uses of each: There is use of the Election class to access the interface and input commands. generateReport is used to create the audit report.
UC_006	User views help window	User inputs help command into the user interface for the command choice. This then triggers the help window to open.
UC_007	Election results are displayed	Classes: Election Methods: runElection() Uses of each: There is use of the Election class to access the interface and input commands. runElection() is used to run the election which will then display results after the election is complete.
UC_008	Program is Tested with command-line arguments	Classes: Election Methods: runElection(), STV(), Plurality() Uses of each: There is use of the Election class to access the interface and input commands. A STV() or a Plurality()

		constructor is created. Then runElection() is called with isTest set to true to execute the test election.
UC_009	Taking in candidate files and ballot file(s)	Classes: Election, Candidate, and Ballot Methods: Candidate(), Ballot() Uses of each: The candidate() and ballot() constructors are used to create objects from the files. Which are then stored as lists in the election class.